PROYECTO INTERMODULAR 2

SPRINT 1 - FASE DE DESARROLLO E IMPLEMENTACIÓN TÉCNICA



Miriam Mendieta Racedo.-
2DAMS
10/10/2026

SPRINT 1 - FASE DE DESARROLLO E IMPLEMENTACIÓN TÉCNICA

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 11 de Octubre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN ---- **1**
RESTRUCTURACIÓN DE HU POR PRIORIDAD (METODOLOGÍA MOSCOW) ---- **2**
RIESGOS Y MITIGACIONES DEL PROYECTO ---- **4**
DIAGRAMAS UML (LENGUAJE UNIFICADO DE MODELADO) ---- **5**
1.Diagrama de Clases ---- 8
2.Diagrama de Actividades (Flujo del Proceso) ---- 10
3.Diagrama de Objetos ---- 11
MODELO ENTIDAD-RELACIÓN (ER) ---- **13**
PASO A TABLAS - NORMALIZACIÓN ---- **16**
CREACIÓN DE LA BASE DE DATOS ---- **18**
ARQUITECTURA DE INICIO Y FLUJO DE NAVEGACIÓN INICIAL ---- **19**
CREACIÓN DEL PROYECTO EN SPRING BOOT CON VS CODE ---- **26**
ESTRUCTURA CORRECTA DE PAQUETES EN SPRING BOOT ---- **33**
ANEXOS ---- **38**
BIBLIOGRAFÍA ---- **39**

INTRODUCCIÓN

Con las bases conceptuales y los requisitos operativos establecidos, iniciamos la Fase de Desarrollo e Implementación Técnica, núcleo práctico de este proyecto de Gestión de Activos Digitales: ***ZeertyDigital.***

Para responder a las necesidades de autónomos y pequeños comercios con bajo perfil técnico, se ha diseñado una arquitectura basada en Java Spring Boot, Thymeleaf y Mysql empaquetada íntegramente en Docker.

Esta estrategia permite aislar por completo el entorno de ejecución, garantizando que la solución pueda desplegarse en la máquina del cliente con un único comando o instalador, sin requerir la configuración de servicios externos ni la instalación previa de Java. A lo largo de esta sección se detalla el desarrollo de la aplicación, el modelado de la persistencia embebida y la creación de la imagen del contenedor para su posterior distribución, asi como también un apartado con la evolución del proceso del trabajo realizado

ZeertyDigital

Restructuración de HU por Prioridad (metodología MoSCoW)

He realizado una reestructuración de las HU aplicando el método MoSCoW. Esto me ayudó a tener una visión realista de la implementación de los módulos y en la gestión del tiempo, ya que dicha metodología junto con Scrum es especialmente útil cuando los plazos son ajustados y es necesario para saber rápidamente sobre lo qué es crítico.

Prioridad	HU incluidas	Estado
MUST (PMV)	HU-01.x, HU-02.x, HU-03.x, HU-04.1, HU-04.3, HU-06.x, HU-07.1	Se desarrollan ahora
SHOULD	HU-04.2, HU-07.2	Se desarrollan después
COULD	Módulo de Marca, Reputación, Activos Tecnológicos. IA	Opcional
WON'T (por ahora)	HU-05.1 y HU-05.2 (OCR y gestoría) Nube	Se implementan más adelante

El PMV de ZeertyDigital se ha redefinido siguiendo la metodología MoSCoW, priorizando únicamente las funcionalidades esenciales para validar el producto y garantizar que cumple su objetivo principal: permitir a cualquier autónomo o pequeño comercio inventariar, clasificar, guardar y respaldar sus activos digitales desde una interfaz centralizada, segura y fácil de usar.

Para ello, se han seleccionado únicamente los módulos que permiten:

- **Acceso seguro a la plataforma**, mediante registro, inicio de sesión y recuperación de contraseña.
- **Inventario centralizado de activos digitales**, con alta, categorización, búsqueda, edición y eliminación.
- **Gestión segura de credenciales**, con cifrado y visualización controlada.
- **Gestión documental básica**, con subida, previsualización y descarga de archivos.
- **Control del negocio**, mediante dashboard y alertas de caducidad.
- **Protección de datos**, mediante generación de copias de seguridad manuales.

Estas funcionalidades permiten validar la utilidad real de la aplicación sin necesidad de módulos avanzados. El PMV garantiza que el usuario puede **organizar y proteger la información crítica del negocio**, evitando pérdidas de accesos, caducidades y dispersión documental. Por ello, se han excluido del PMV funcionalidades complementarias como OCR, exportación trimestral, sincronización con Drive o restauración de backups, que aportan valor, pero no son necesarias para validar el producto.

La reestructuración garantiza:

- Cumplimiento de fechas de entrega
- Un PMV sólido y demostrable
- Un vídeo final atractivo con mejoras reales
- Un desarrollo escalonado y profesional
- Una planificación alineada con MoSCoW y con la carga de trabajo realista

Funcionalidades obligatorias para que la app pueda usarse:

- HU-01.1: Acceso al Sistema → Inicio y Cierre de Sesión (Must Have - Requisito de Entrada)
  Permite a cualquier usuario validarse con su correo/contraseña para entrar al panel y cerrar sesión de forma segura al finalizar.
- HU-01.2: Registro Autónomo de Usuario → Autorregistro (Must Have - Según el modelo de negocio)
  Permite a un nuevo dueño/autónomo crear su cuenta inicial (ROLE_ADMIN)
- HU-01.3: Gestión y Alta de Usuarios por Administrador (Must Have - Control Interno)
  Permite al Administrador (titular de la PYME o personal autorizado) invitar o dar de alta a sus empleados y asignarles su rol específico (UsuarioApp, Visitante, Administradoretc.).
- HU-06.1 Dashboard Base: Es la pantalla principal tras iniciar sesión → Must Have.

Todas estas tareas son **Must Have** porque sin ellas no existe arquitectura, base de datos ni backend funcional.

Riesgos y Mitigaciones del Proyecto

La mitigación es el conjunto de acciones preventivas o correctivas que se aplican para reducir el impacto de un riesgo o evitar que ocurra durante el desarrollo del proyecto.

*** Riesgo 1 — Sobrecarga de funcionalidades:** Intentar incluir OCR o módulos avanzados. **Mitigación:** Reestructuración MoSCoW → OCR pasa a mejora futura.

*** Riesgo 2 — Dependencias externas (Google Drive, OCR, APIs):** Integraciones que pueden fallar o requerir más tiempo. **Mitigación:** Mantener PMV sin dependencias externas.

*** Riesgo 3 — Complejidad del cifrado y seguridad:** Implementación incorrecta de cifrado puede comprometer datos. **Mitigación:** Sprint 3 dedicado exclusivamente a seguridad.

*** Riesgo 4 — Retrasos en la entregas hasta llegar al vídeo (aprox. 10/11):** Si el PMV no está estable o las entregas se retrasan por una mala gestión del tiempo, el vídeo no puede grabarse. **Mitigación:** PMV completo en Sprint 3 → Sprint 4 solo para vídeo.

***Riesgo 5 — Falta de datos reales para pruebas:** Sin activos reales, el sistema no se valida correctamente. **Mitigación:** Checklist de auditoría inicial integrada en el documento.

*** Riesgo 6 — Problemas de integración entre módulos:** Fallos al unir usuarios, activos, credenciales y documentos. **Mitigación:** Sprint 4 dedicado a pruebas de integración.

Diagramas UML (Lenguaje Unificado de Modelado)

El uso de diagramas UML proporciona un lenguaje visual estandarizado que permite representar de forma clara la estructura, el comportamiento y las interacciones de un sistema. Incorporar esta modelación en este momento del desarrollo aporta orden, claridad y previsión. Esto se traduce en un menor margen de error, la reducción de costes de mantenimiento y una base documental sólida que actúa como puente entre la arquitectura conceptual y la implementación en código, garantizando un software más robusto, coherente y fácil de evolucionar.

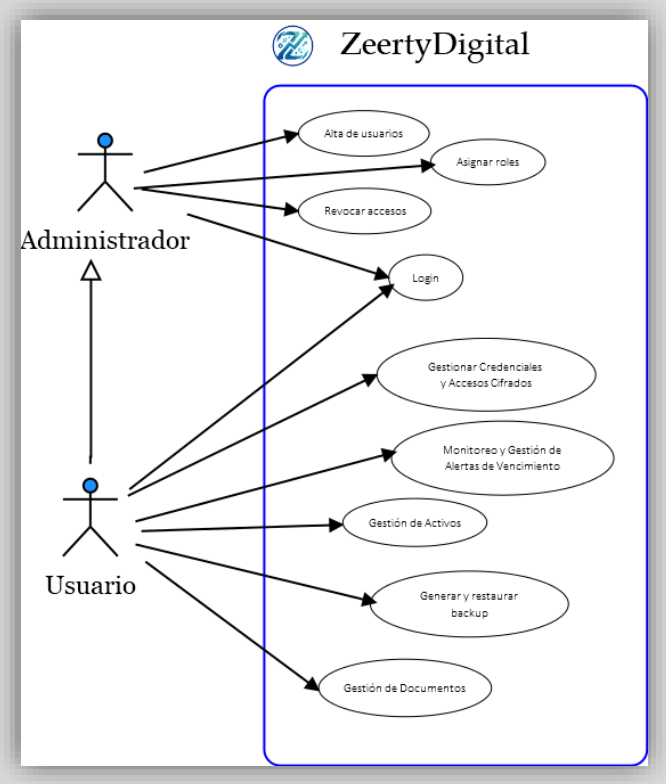


Diagrama de Casos de Uso

Especificación detallada de los casos de uso del Sprint 1

CASO DE USO:	Acceso y Cierre de Sesión
Descripción	Permite a los usuarios registrados autenticarse en el sistema con correo y contraseña cifrada, así como finalizar su sesión de forma segura (*Must Have*).
Actores	Usuario Registrado / Usuario no Autenticado
Precondición	El usuario debe estar registrado previamente en la base de datos MySQL.
Flujo principal	1. El usuario accede al formulario de login (/login). 2. Introduce su correo y contraseña. 3. El sistema (Spring Security) valida las credenciales contrastando el hash BCrypt. 4. El sistema inicia la sesión y redirige al Dashboard según el rol del usuario. 5. Para salir, el usuario pulsa "Cerrar Sesión" e invalida el contexto de seguridad.
Flujos alternativos	Credenciales incorrectas: muestra el mensaje *"Correo o contraseña no válidos"*.
Postcondición	El usuario accede al sistema con sus permisos asignados o destruye su sesión de forma segura.

CASO DE USO:	Alta y Gestión de Usuarios por Administrador
Descripción	Permite al titular/administrador dar de alta a nuevos usuarios y asignarles un rol específico (RBAC) para limitar su acceso a la información (*Must Have*).
Actores	Administrador (ROLE_ADMIN)
Precondición	El administrador debe estar autenticado con permisos de gestión en la plataforma.
Flujo principal	1. El administrador accede a la sección "Gestión de Usuarios". 2. Pulsa en "Crear Nuevo Usuario" y rellena el formulario (Nombre,Apellidos, Email, Password temporal y Rol). 3. El sistema valida que el correo no esté repetido. 4. El sistema guarda la entidad UsuarioApp asociándole la entidad Rol seleccionada. 5. El nuevo usuario aparece activo en el listado general.
Flujos alternativos	Correo duplicado: muestra el mensaje *"El usuario ya existe en el sistema"*.
Postcondición	Se crea la nueva cuenta de usuario con sus restricciones y roles correspondientes.

1. Diagrama de Clases

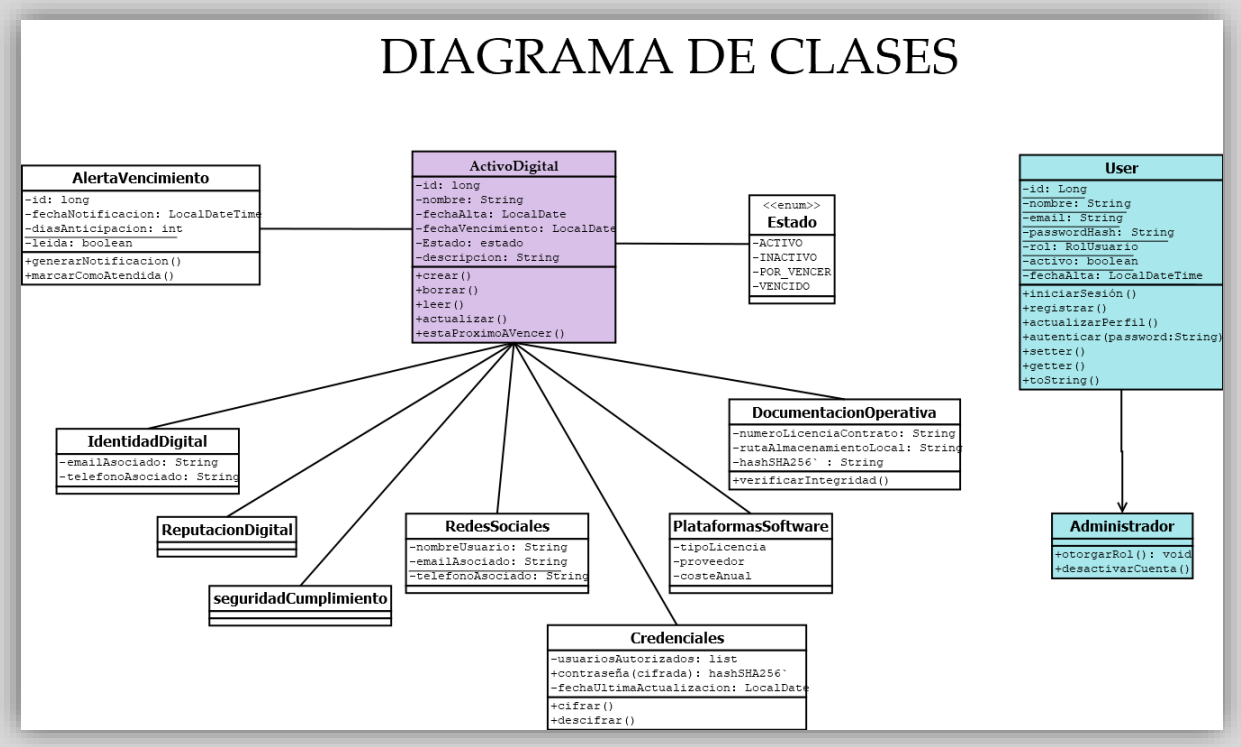
Representa la arquitectura estática de la app, se modela el dominio del problema traduciendo los requerimientos del negocio en entidades de software, definiendo sus atributos, métodos principales, modificadores de acceso y las relaciones de asociación, herencia y composición entre ellas.

Clases de Entidad

- ***UsuarioApp:*** Representa al usuario registrado o administrador del sistema.
- ***ActivoDigital:*** Entidad central del inventario (recurso registrado).
- ***Categoria*:** Clasificación del activo digital (Software, Redes Sociales, Dominios, etc.).
- ***Documento****:* Ficheros adjuntos.
- ***Etiqueta (Tag):*** Esencial para los motores de búsqueda internos. Mantiene una relación de *Muchos a Muchos* con los activos (un activo puede tener muchas etiquetas, y una etiqueta puede pertenecer a muchos activos)
- ***Backup:*** Registro de las copias de seguridad generadas e importadas.

Enumerados / Enums

- ***EstadoActivo***: Estado del activo (ACTIVO, POR_VENCER, VENCIDO, INACTIVO).
- ***Etiquetas***: MARKETING, LEGAL, FINANZAS, TECNOLOGIA, RGPD_DATOS, CONFIDENCIAL, ACCESO_PUBLICO
- ***Categoria***: Clasificación del activo digital (Software, Redes Sociales, Dominios, etc.).



Relaciones y Cardinalidad entre Entidades

- Usuario—— (1 : N) —— ActivoDigital(Composición) → Un usuario posee de cero a múltiples (0..*`) activos digitales. Cada activo digital pertenece de manera unívoca (1) a un único usuario.

- ActivoDigital —— (1 : N) —— AlertaVencimiento (Asociación Simple) → Un activo con fecha de expiración puede programar múltiples alertas temporales (0..*) a medida que se aproximan sus fechas límites (ej. a 30 días, 15 días y 3 días de la expiración).

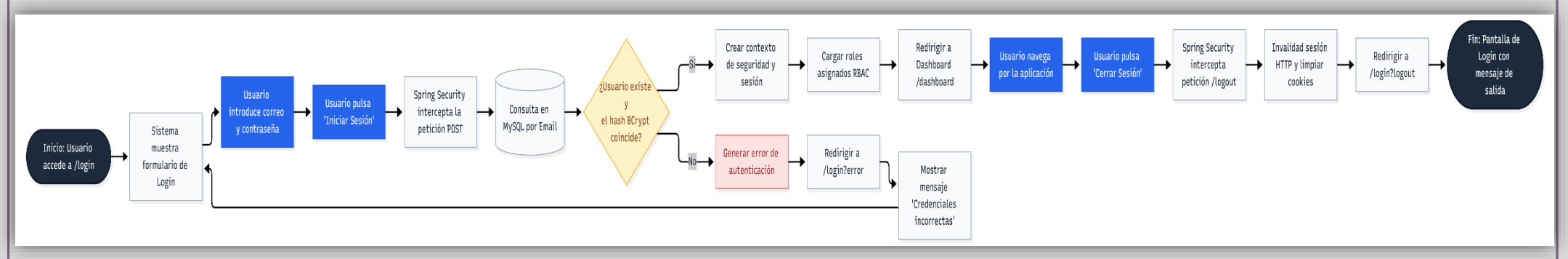
2. Diagrama de Actividades (Flujo del Proceso)

Este flujo describe la secuencia lógica de pasos, decisiones y estados desde que el usuario interactúa con la pantalla de autenticación hasta que finaliza su sesión activa dentro de la aplicación.

- **Flujo de Inicio de Sesión:**
  1. El usuario solicita acceder al sistema mediante la ruta /login.
  2. El sistema renderiza la vista HTML (Thymeleaf).
  3. El usuario envía sus credenciales (correo electrónico y contraseña).
  4. **Punto de Decisión (Spring Security):** Se consulta la base de datos MySQL y se compara la contraseña usando BCryptPasswordEncoder.
     - **Credenciales Inválidas:** Se genera un mensaje de error y se redirige de nuevo al formulario.
     - **Credenciales Válidas:** Se establece el contexto de seguridad (Session/Principal) y se redirige al Dashboard según el rol del usuario (ROLE_ADMIN, ROLE_USUARIO, ROLE_AUDITOR etc.).

- **Flujo de Cierre de Sesión:**
  1. El usuario pulsa la opción *"Cerrar Sesión"* en el menú de navegación.
  2. Spring Security intercepta la petición /logout.
  3. Se invalida la sesión HTTP y se limpian las cookies/contexto de seguridad.
  4. El sistema redirige a la vista pública de inicio de sesión con el mensaje de confirmación de salida.

Diagrama de Actividades



3. Diagrama de Objetos

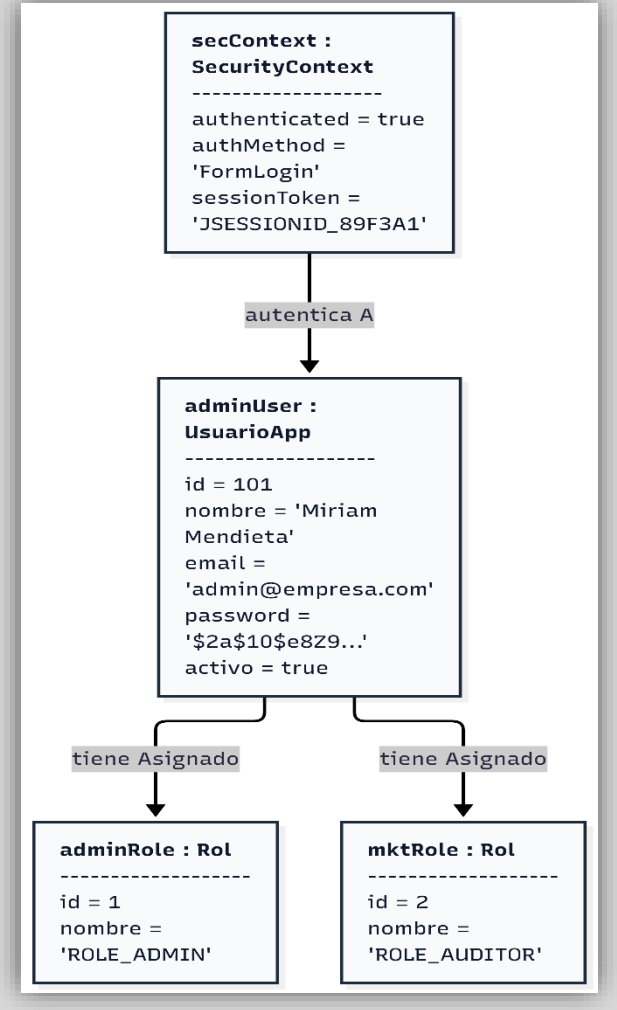
Un Diagrama de Objetos (UML) representa una fotografía o instancia concreta en un momento específico de la ejecución del sistema, mostrando los objetos reales creados en memoria y los valores de sus atributos.

Para el Sprint 1 (centrado en la autenticación, carga de datos iniciales y asignación de roles RBAC), el diagrama de objetos más representativo es el estado de la memoria cuando el Administrador se ha autenticado en la aplicación.

Estado representado en el diagrama
En este momento concreto de ejecución existen:

- Dos instancias de la clase Rol: ROLE_ADMIN y ROLE_AUDITOR.
- Una instancia de UsuarioApp (el Administrador/Titular de la PYME) vinculado a sus roles.
- El objeto de sesión de seguridad en Spring Security (SecurityContext) que valida que dicho usuario está autenticado en la app.

PROYECTO INTERMODULAR 2



Modelo Entidad-Relación (ER)

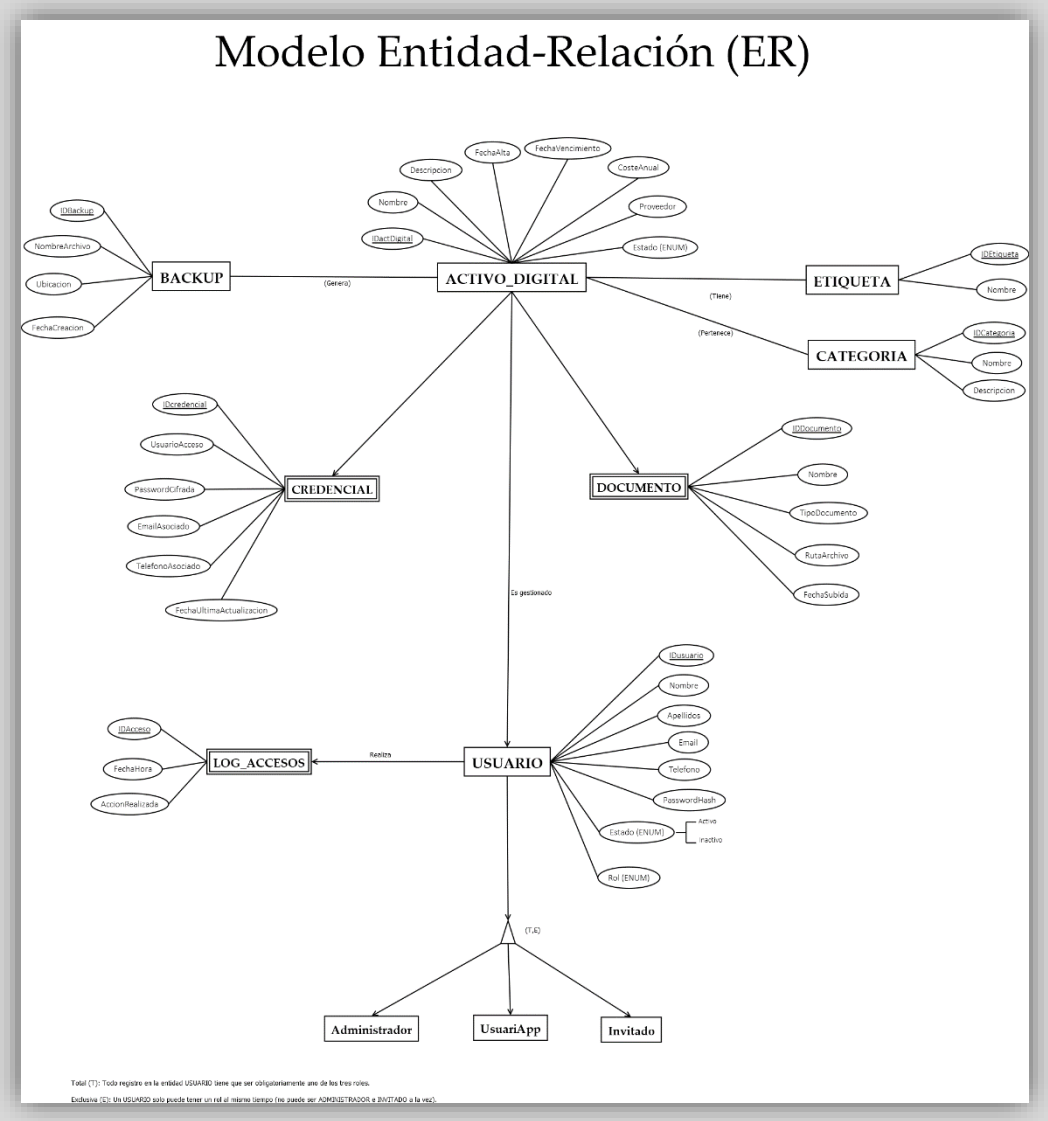
Es una representación visual de cómo se organizan y conectan los datos dentro de un sistema. Su importancia radica en que permite estructurar la información de forma clara, evitar duplicidades (redundancia) y asegurar la integridad de la base de datos antes de su implementación física.

El Modelo ER define la estructura lógica de los datos del sistema, permitiendo:

- Organizar los activos digitales y recursos del negocio de forma coherente.
- Garantizar la integridad referencial y las relaciones entre entidades.
- Facilitar el diseño de operaciones CRUD y el rendimiento de consultas eficientes.
- Preparar la estructura base para la creación física del esquema en MySQL.

El sistema deberá gestionar:

- Usuarios y roles
- Activos digitales
- Categorías
- Estados
- Documentos asociados
- Credenciales cifradas
- Copias de seguridad (Backups)


Modelo Entidad-Relación (ER)
BACKUP
IDBackup
NombreArchivo
Ubicacion
FechaCreacion
(Genera)
ACTIVO_DIGITAL
IDActDigital
Nombre
Descripcion
FechaAlta
FechaVencimiento
CosteAnual
Proveedor
Estado (ENUM)
(Tiene)
ETIQUETA
IDEtiqueta
Nombre
(Pertenece)
CATEGORIA
IDCategoria
Nombre
Descripcion
CREDENCIAL
IDcredencial
UsuarioAcceso
PasswordCifrada
EmailAsociado
TelefonoAsociado
FechaUltimaActualizacion
DOCUMENTO
IDDocumento
Nombre
TipoDocumento
RutaArchivo
FechaSubida
Es gestionado
LOG_ACCESOS
IDAcceso
FechaHora
AccionRealizada
Realiza
USUARIO
IDusuario
Nombre
Apellidos
Email
Telefono
PasswordHash
Estado (ENUM)
Activo
Inactivo
Rol (ENUM)
(T,E)
Administrador
UsuariApp
Invitado
Total (T): Todo registro en la entidad USUARIO tiene que ser obligatoriamente uno de los tres roles.
Exclusiva (E): Un USUARIO solo puede tener un rol al mismo tiempo (no puede ser ADMINISTRADOR e INVITADO a la vez).

La entidad USUARIO se especializa en tres subtipos: ADMINISTRADOR, USUARIO_APP e INVITADO. Esta jerarquía cumple con las siguientes restricciones de cobertura:

- **Especialización Total (T):** Todo registro existente en la entidad USUARIO debe pertenecer obligatoriamente a uno de los tres subtipos. No pueden existir usuarios sin un rol asignado.
- **Especialización Exclusiva / Disjunta (E):** Los subtipos son mutuamente excluyentes. Un USUARIO solo puede poseer un único rol simultáneamente (un usuario no puede ser ADMINISTRADOR e INVITADO al mismo tiempo).

(Nota: Al no contar los subtipos con atributos propios adicionales, en la fase de paso al Modelo Relacional se unificarán en la tabla principal USUARIOS mediante la columna Rol ENUM('ADMINISTRADOR', 'USUARIO_APP', 'INVITADO') VNN).

Relaciones:

- Usuarios → activos_digitales (1:N): Un usuario registrado administra o crea varios activos digitales.
- Categorías → activos_digitales (1:N): Cada activo pertenece a una única categoría principal.
- activos_digitales →credenciales (1:N): Un activo digital puede tener una o varias credenciales de acceso asociadas.
- activos_digitales → documentos (1:N): Un activo digital puede adjuntar múltiples archivos/documentos operativos.
- activos_digitales ←→ etiquetas (N:M): Resuelto mediante la tabla intermedia activo_etiqueta.
- usuarios → backups (1:N): Cada copia de seguridad queda auditada con el usuario que la generó o importó.

Paso a Tablas - Normalización

La BD solo va a guardar a los usuarios logueados en la app.

USUARIOS (IDUsuario, Nombre, Apellidos, Email, PasswordHash, Telefono, Rol, Estado)
PK: {IDUsuario}

CATEGORIAS (IDCategoria, Nombre, Descripcion)
PK: {IDCategoria}

ACTIVOS_DIGITALES (IDActivo, Nombre, Descripcion, Estado, Proveedor, , CosteAnual, FechaAlta, FechaVencimiento, IDCategoria, IDUsuario)
PK: {IDActivo}
FK: {IDCategoria} → CATEGORIAS({IDCategoria}) (BR/MC) VNN
FK: {IDUsuario} → USUARIOS({IDUsuario}) (BR/MC) VNN

CREDENCIALES (IDCredencial, UsuarioAcceso, PasswordCifrada, EmailAsociado, TelefonoAsociado, FechaUltimaActualizacion, IDActivo)
PK: {IDCredencial}
FK: {IDActivo} → ACTIVOS_DIGITALES({IDActivo}) (BR/MC) VNN

DOCUMENTOS (IDDocumento, Nombre, TipoDocumento, RutaArchivo, FechaSubida, IDActivo)
PK: {IDDocumento}
FK: {IDActivo} → ACTIVOS_DIGITALES({IDActivo}) (BR/MC) VNN

ETIQUETAS (IDEtiqueta, Nombre)
PK: {IDEtiqueta}

ACTIVO_ETIQUETA (IDActivo, IDEtiqueta)

PK: {IDActivo, IDEtiqueta}
FK: {IDActivo} → ACTIVOS_DIGITALES({IDActivo}) (BR/MC) VNN
FK: {IDEtiqueta} → ETIQUETAS({IDEtiqueta}) (BR/MC) VNN

BACKUPS (IDBackup, NombreArchivo, Tipo, Ubicacion, FechaCreacion, IDUsuario)

PK: {IDBackup}
FK: {IDUsuario} → USUARIOS({IDUsuario}) (BR/MC) VNN

LOG_ACCESOS (IDLog, FechaHora, TipoAccion, IDUsuario, IDActivo)

PK: {IDLog}FK: {IDUsuario} → USUARIOS({IDUsuario}) (BR/MC) VNN
FK: {IDActivo} →ACTIVOS_DIGITALES({IDActivo}) (BR/MC)

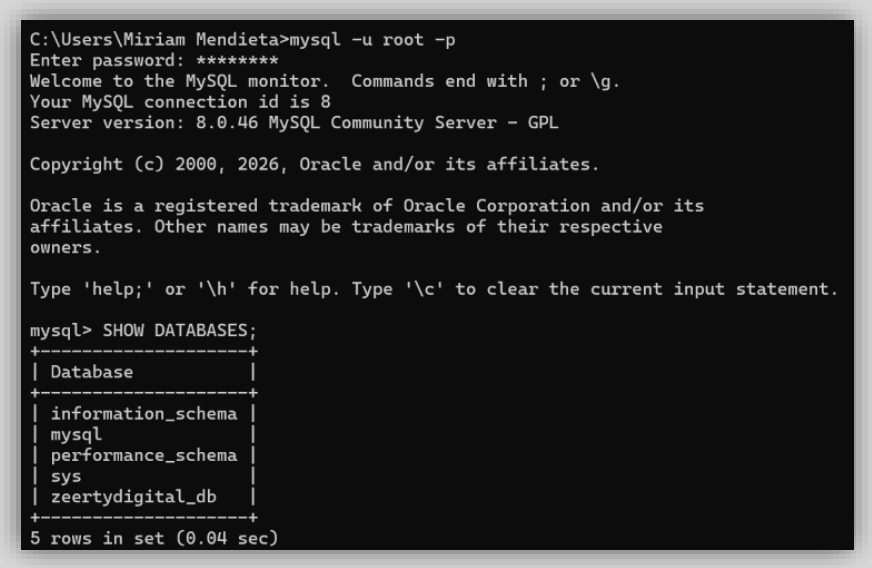
Glosario de Nomenclatura Utilizada

- PK: Primary Key (Clave Primaria).
- FK: Foreign Key (Clave Foránea).
- BR/MC: Borrado en Regla / Modificación en Cascada (o Borrado Restringido / Modificación en Cascada según la nomenclatura de tu centro académico).
- VNN: Valor No Nulo (NOT NULL).
- En credenciales el “usuarioAcceso” es Nombre de usuario / login para iniciar sesión en ese activo digital.

Creación de la Base de Datos

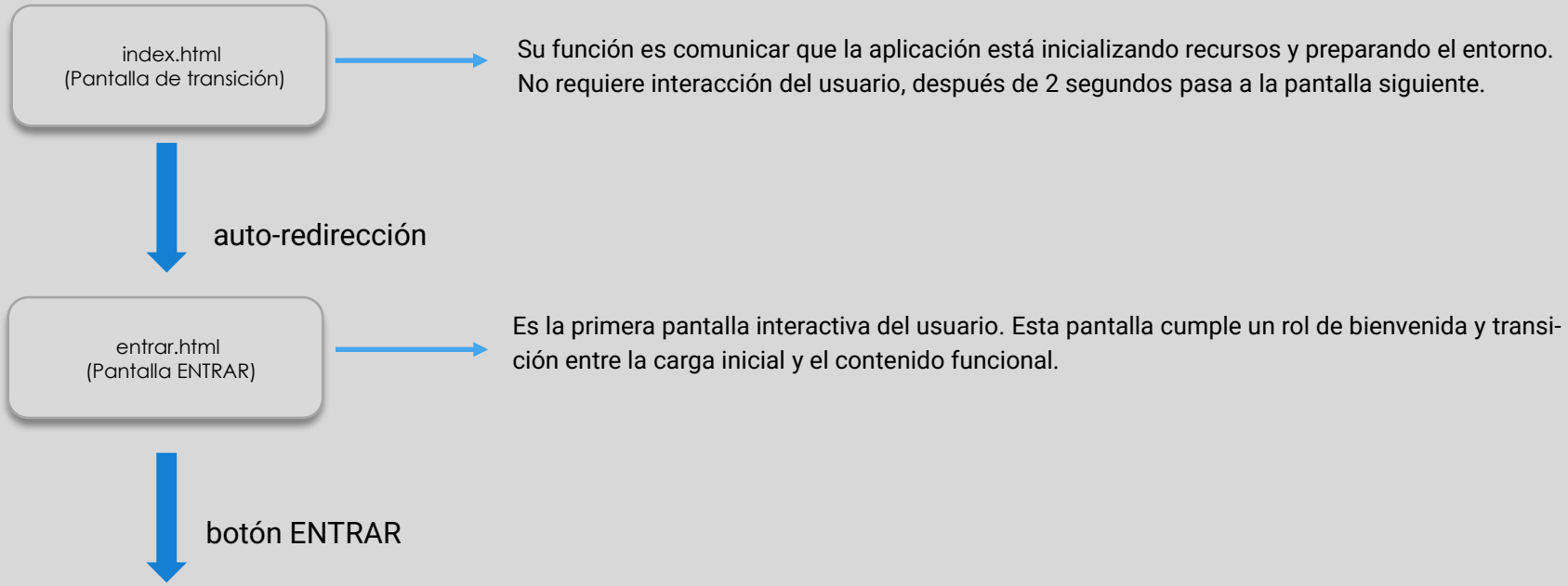
Se creo la base de datos desde MySQL Workbench: tecla Windows → Escribir MySQL Workbench → entrar con contraseña si se solicita → CREATE DATABASE zeertydigital_db; → Busca en la barra de herramientas icono de un rayo amarillo solo (sin lupas ni carpetas), clicar

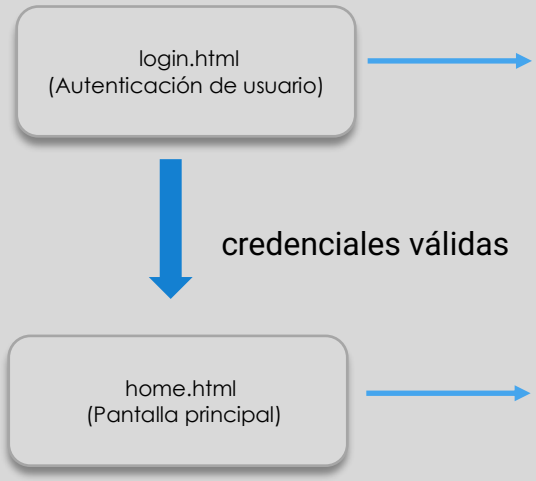
Comprobación por consola de la creación de la base de datos



Arquitectura de Inicio y Flujo de Navegación Inicial

Definición del comportamiento del sistema desde el momento en que el usuario accede a la aplicación ***ZeertyDigital***, detallando las pantallas iniciales, el punto de entrada, la secuencia de navegación y el flujo de autenticación requerido para completar el Sprint correspondiente a la entrada de usuario.





La pantalla login.html es el componente central del Sprint de autenticación. Su objetivo es permitir que el usuario acceda al sistema mediante credenciales válidas.

Aquí se encuentra el contenido real de la aplicación. Su estructuración e implementación se realizará en el próximo Spring.

En el caso de los Admin al loguearse también podrán entrar a la pantalla de gestión de usuarios.

Técnica del Flujo de pantallas

- Separación clara de responsabilidades → Cada pantalla cumple un rol específico: carga, bienvenida, autenticación y contenido.
- Cumplimiento del Sprint → La pantalla de login satisface el requisito de “entrada de usuario” dentro del Sprint 1.
- Escalabilidad → El flujo permite añadir recuperación de contraseña, registro, verificación por correo, roles, etc.
- Experiencia de usuario optimizada → La transición inicial y la pantalla de entrada suavizan el acceso al login.
- Compatibilidad responsiva → web, móvil, híbrido.

PROYECTO INTERMODULAR 2

Estructura de carpetas de mis archivos estáticos creados en principio

```
Zeerty-app/
        |
        ├── index.html            # Pantalla inicial
        ├── entrar.html           # Pantalla de transición
        ├── login.html            # Pantalla de inicio de sesión
        ├── home.html             # Pantalla principal (contenido real)
        |
        ├── css/
        |       ├── global.css          # Estilos globales
        |       ├── login.css           # Estilos del login
        |       ├── transition.css      # Estilos de la pantalla index
        |       ├── home.css            # Estilos de la pantalla home
        |       └── entrar.css          # Estillo pantalla de entrada
        |
        └── img/
                ├── logo-zeerty.png     # Logo principal
                └── icons/              # Iconos del sistema
```

PROYECTO INTERMODULAR 2

index.html → modificado para Thymeleaf

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<!--
xmlns:th="http://www.thymeleaf.org"
→ Activa el “espacio de nombres” de Thymeleaf.
→ Permite usar atributos especiales como th:href, th:src, th:text, th:each, etc.
→ Sin esto, Thymeleaf no reconoce sus propias funciones.
-->

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <meta content="Miriam Mendieta" name="author">
  <title>ZeertyDigital - Tecnología que Acompaña, Simplifica y Avanza</title>
  <meta name="description"
    content="Soluciones tecnológicas avanzadas para simplificar, proteger y hacer avanzar tu entorno digital.">

  <meta name="robots" content="index, follow">
  <meta name="googlebot" content="index, follow">

  <meta name="theme-color" content="#232a40">
```

PROYECTO INTERMODULAR 2

```
  <!-- Favicon -->
  <!--
  th:href="@{/ruta}"
  → Thymeleaf genera la ruta correcta hacia /static/img/...
  → Evita errores cuando cambias el puerto o despliegas en servidor.
  -->
  <link rel="apple-touch-icon" sizes="180x180" th:href="@{/img/favicon_io/apple-touch-icon.png}">
  <link rel="icon" type="image/png" sizes="32x32" th:href="@{/img/favicon_io/favicon-32x32.png}">
  <link rel="icon" type="image/png" sizes="16x16" th:href="@{/img/favicon_io/favicon-16x16.png}">
  <link rel="manifest" th:href="@{/img/favicon_io/site.webmanifest}">

  <!-- Preload -->
  <!--
  th:href="@{/css/global.css}"
  → Thymeleaf resuelve la ruta al archivo dentro de /static/css/

  as="style"
  → Indica al navegador que el recurso es CSS.
  -->
  <link rel="preload" th:href="@{/css/global.css}" as="style">
  <link rel="preload" th:href="@{/css/transition.css}" as="style">
  <link rel="preload" th:href="@{/img/logoZeerty.png}" as="image">

  <!-- CSS -->
  <link rel="stylesheet" th:href="@{/css/global.css}">
  <link rel="stylesheet" th:href="@{/css/transition.css}">
</head>
```

PROYECTO INTERMODULAR 2

```
<body>
  <div id="loader-wrapper">
    <div class="loader-content">
      <!--
      th:src="@{/img/logoZeerty.png}"
      → Thymeleaf genera la ruta correcta hacia la imagen.
      -->
      <img th:src="@{/img/logoZeerty.png}" alt="ZeertyDigital" class="loader-logo">
      <div class="loading-spinner"></div>
      <p class="loading-text">CARGANDO...</p>
    </div>
  </div>

  <!-- Redirige a la página de inicio de sesión después de 3 segundos -->
  <script>
    setTimeout(() => {
      window.location.href = "/entrar";
    }, 3000);
  </script>
```

Los archivos CSS no requieren modificaciones internas cuando se usa Thymeleaf.

El único cambio necesario es en el HTML, donde las rutas deben declararse con th:href="@{/css/archivo.css}" para que Thymeleaf genere correctamente las rutas hacia los recursos estáticos.



Creación del proyecto en Spring Boot con VS Code

El proyecto se estructura en tres capas: **Frontend estático (HTML/CSS/JS)** servido desde /static, **Backend Spring Boot** que expone una API REST, y **Persistencia con JPA** conectada a una base de datos MySQL creada previamente en Workbench. El frontend se comunica con el backend mediante fetch(), y el backend gestiona las operaciones CRUD sobre la base de datos. Esta arquitectura desacopla presentación, lógica y datos, mejora la seguridad y facilita la escalabilidad.

- Instalación de extensiones → Extension Pack for Java, Spring Boot Extension Pack.
- Crea un proyecto: Ctrl + Shift + P → Spring Initializr: Create a Maven Project
- Seleccionar: nombre de paquete(com.miriammendieta), nombre de la app(zeerty_digital_app), java 25, Spring Boot 4.x
  Configurar dependencias en pom.xml:

```
<dependencies>
    <!-- Dependencia de Thymeleaf para los archivos HTML -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>

    <!-- Conector oficial de MySQL -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
  </dependencies>
```

PROYECTO INTERMODULAR 2

- Crear los controladores que sirven para las vistas Thymeleaf:

```java
package com.miriammendieta.zeerty_digital_app.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class ViewController {

    @GetMapping("/")
    public String inicio() {
        return "index"; // index.html en /templates
    }

    @GetMapping("/entrar")
    public String entrar() {
        return "entrar"; // entrar.html
    }

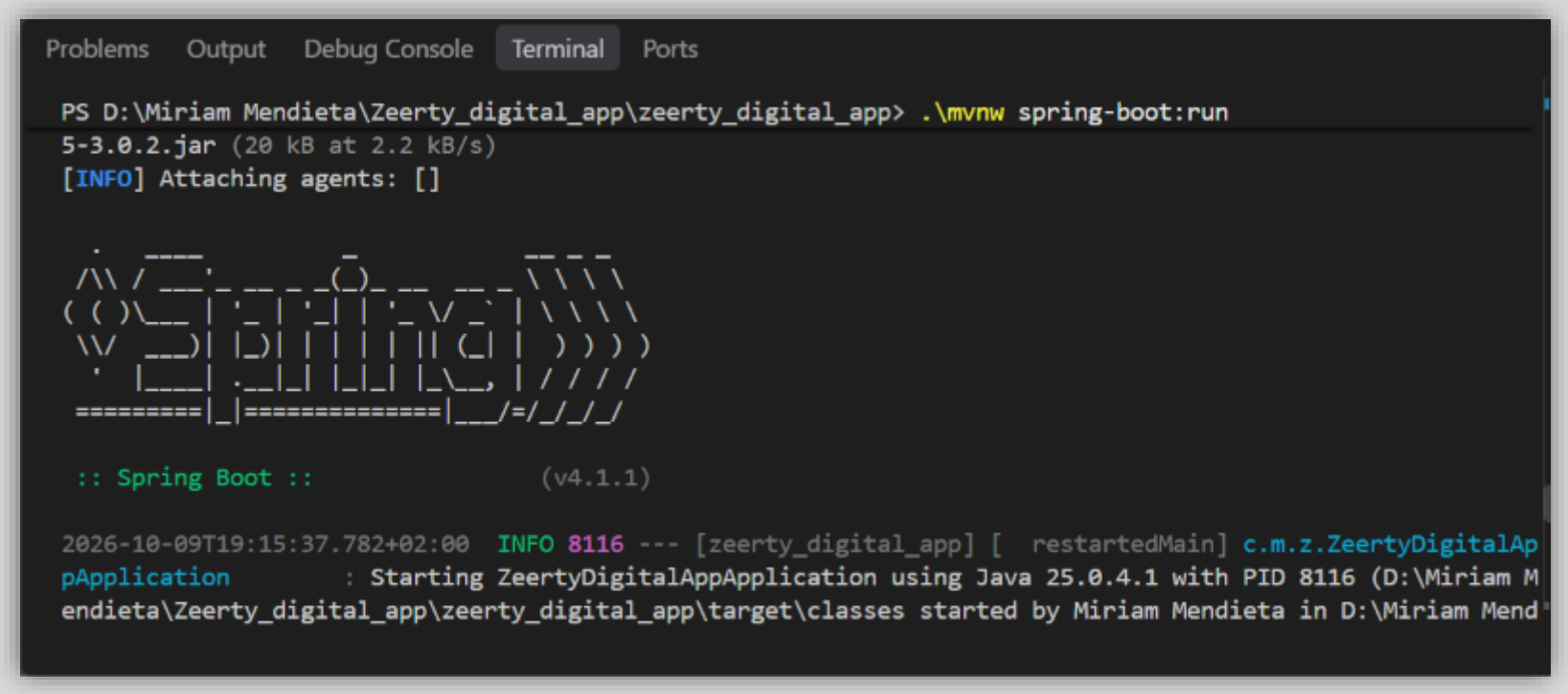
    @GetMapping("/login")
    public String login() {
        return "login"; // login.html
    }

    @GetMapping("/home")
    public String home() {
        return "home"; // home.html
    }
}
```

PROYECTO INTERMODULAR 2

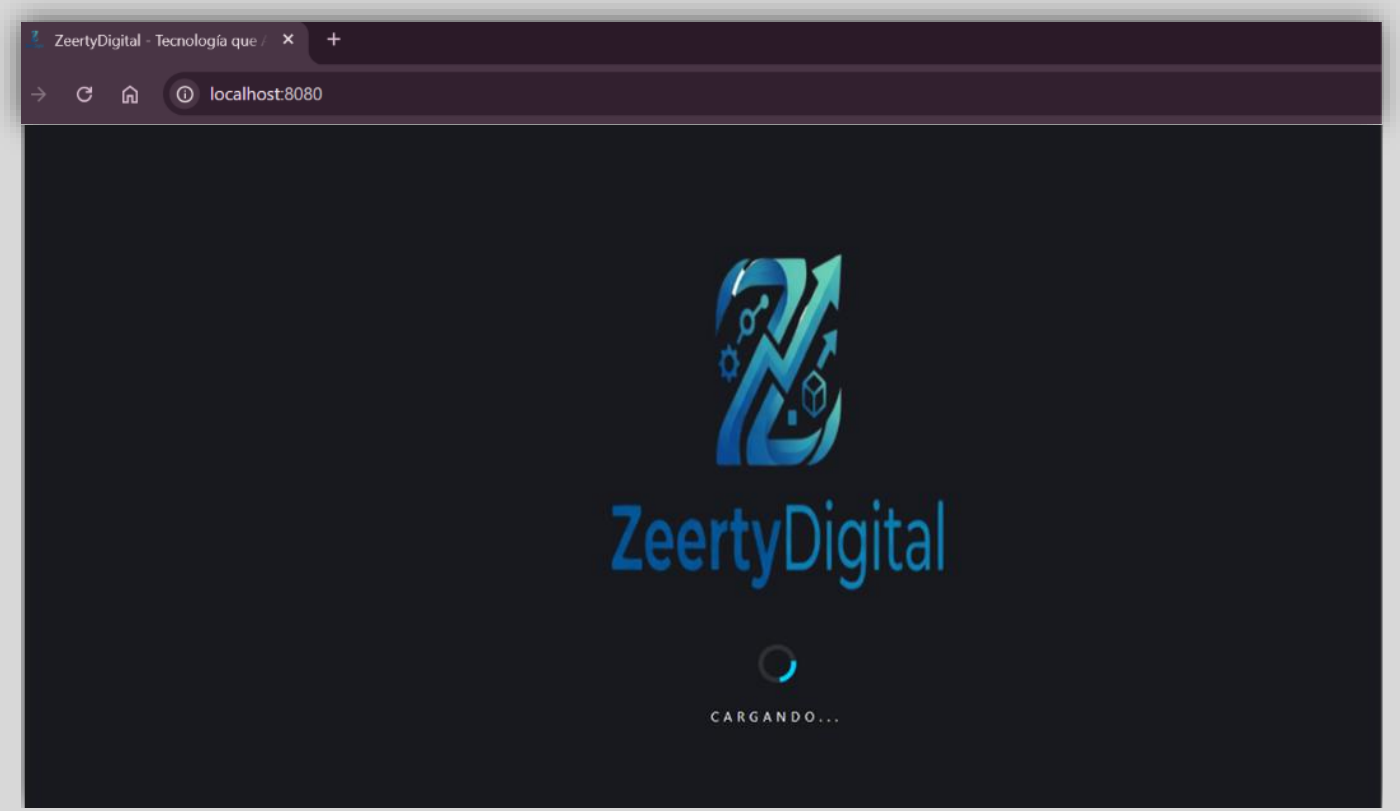
El paquete controller debe crearse dentro del paquete raíz del proyecto, ubicado en src/main/java. Dentro de este paquete se definen los controladores Spring Boot que gestionan las rutas (@GetMapping) y cargan las vistas Thymeleaf ubicadas en src/main/resources/templates. Sin este paquete y sus controladores, las vistas no pueden ser renderizadas por Spring Boot.
La carpeta /templates **no funciona sola**, necesita un controlador que diga: “Cuando el usuario entre a /login, carga login.html”.

- Levantar el Servidor desde la terminal → .\mvnw spring-boot:run
  Si hay algún error, solucionarlo y luego limpiar la consola con → .\mvnw clean

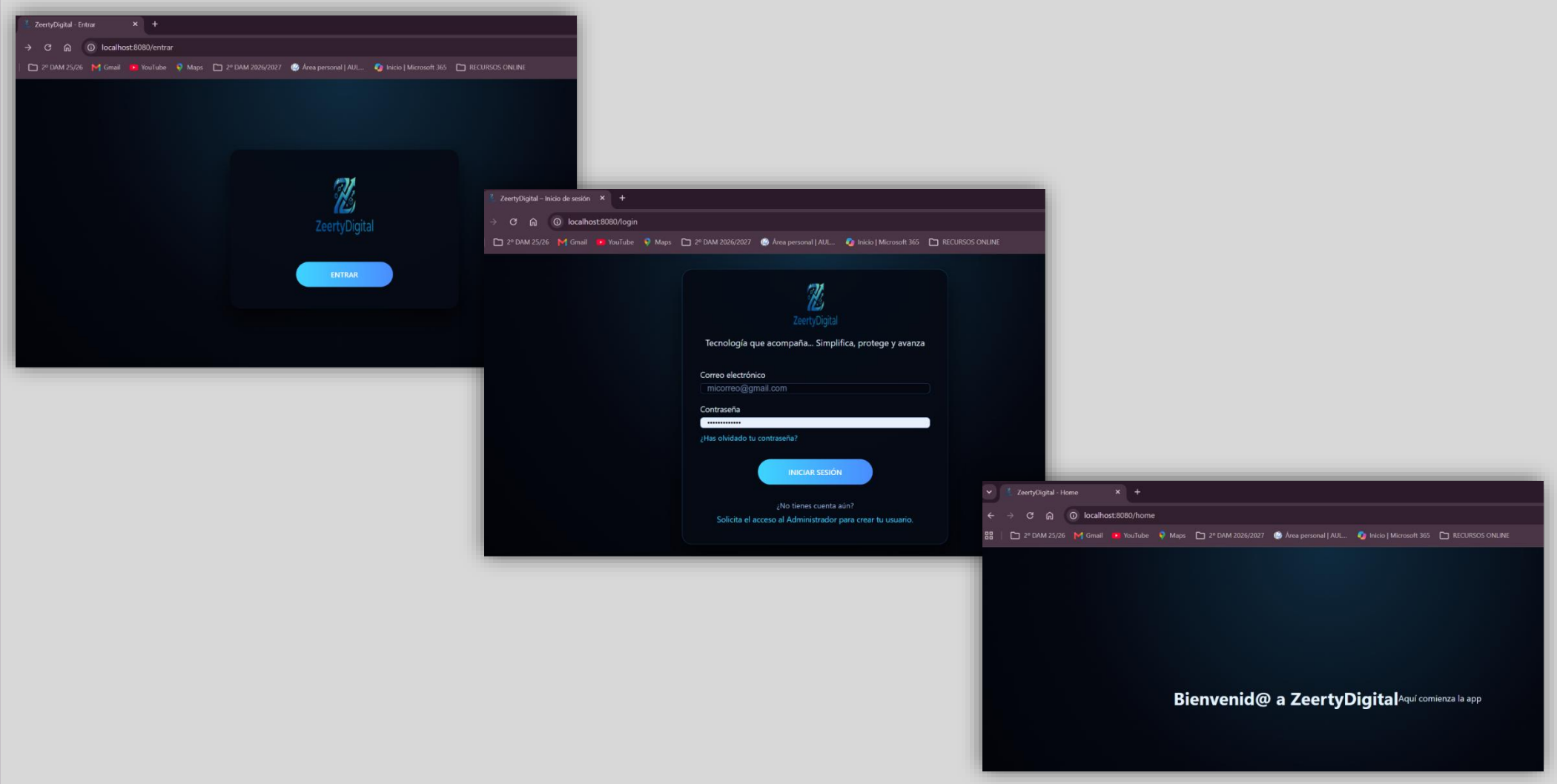


PROYECTO INTERMODULAR 2

En el navegador entrar a → http://localhost:8080/



PROYECTO INTERMODULAR 2



PROYECTO INTERMODULAR 2

- Configurar la conexión a tu base de datos MySQL → En application.properties:

```
spring.application.name=zeerty_digital_app

spring.datasource.url=jdbc:mysql://localhost:3306/zeertyDigital_db
spring.datasource.username=root
spring.datasource.password=TU PASSWORD

spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

Si MySQL está en otro puerto o el puerto esta ocupado se cambia (por ejemplo 3307):

jdbc:mysql://localhost:3307/NOMBRE de BD

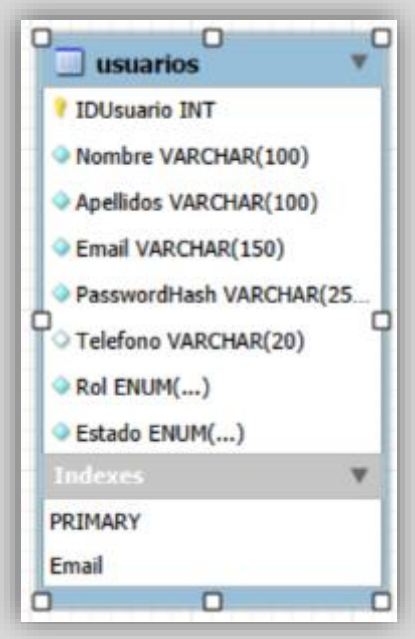
- Conectarse al Workbench para crear la BD y la tabla usuario

CREATE DATABASE zeertyDigital_db;

PROYECTO INTERMODULAR 2

Tabla: USUARIOS

```
CREATE TABLE USUARIOS (
    IDUsuario INT AUTO_INCREMENT,
    Nombre VARCHAR(100) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    Telefono VARCHAR(20),
    Rol ENUM('ADMINISTRADOR', 'USUARIO_APP', 'INVITADO') NOT NULL DEFAULT 'USUARIO_APP',
    Estado ENUM('ACTIVO', 'INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    PRIMARY KEY (IDUsuario)
) ENGINE=InnoDB;
```



Ingreso un usuario de prueba:

```
INSERT INTO USUARIOS (Nombre,Apellidos, Email, PasswordHash, Telefono, Rol, Estado)
VALUES ('Miriam','Mendieta Racedo', 'miriam@example.com', '1234', '600000000', 'USUARIO_APP', 'ACTIVO');
```

Estructura correcta de paquetes en Spring Boot

En un proyecto bien organizado, normalmente se tiene:

src/main/java/com/tuempresa/tuapp/

controller/ → Aquí van las clases que reciben peticiones del navegador (rutas, formularios, vistas. Ejemplo: LoginController).

model/ → Aquí van las entidades JPA, es decir, las clases que representan las tablas de MySQL (usuario.java, Estado.java).

repository/ → Aquí van las interfaces que conectan con la base de datos usando JPA (UsuarioRepository.java).

service/ → Aquí va la lógica de negocio: validaciones, login, reglas, etc (UsuarioService.java).

- Conectar con Spring Boot.
  - Crear paquete model →btn derecho → new folder → model
  - Dentro de model → new file → usuario.java
    Rol.java
    Estado.java

  - Crear el paquete controller →LoginController.java

    El controlador recibe los datos del formulario HTML y llama al servicio.

    Función principal:

    - Recibir email y password
    - Llamar a usuarioService.login()
    - Redirigir a home.html o devolver error

- Probar el login desde el navegador

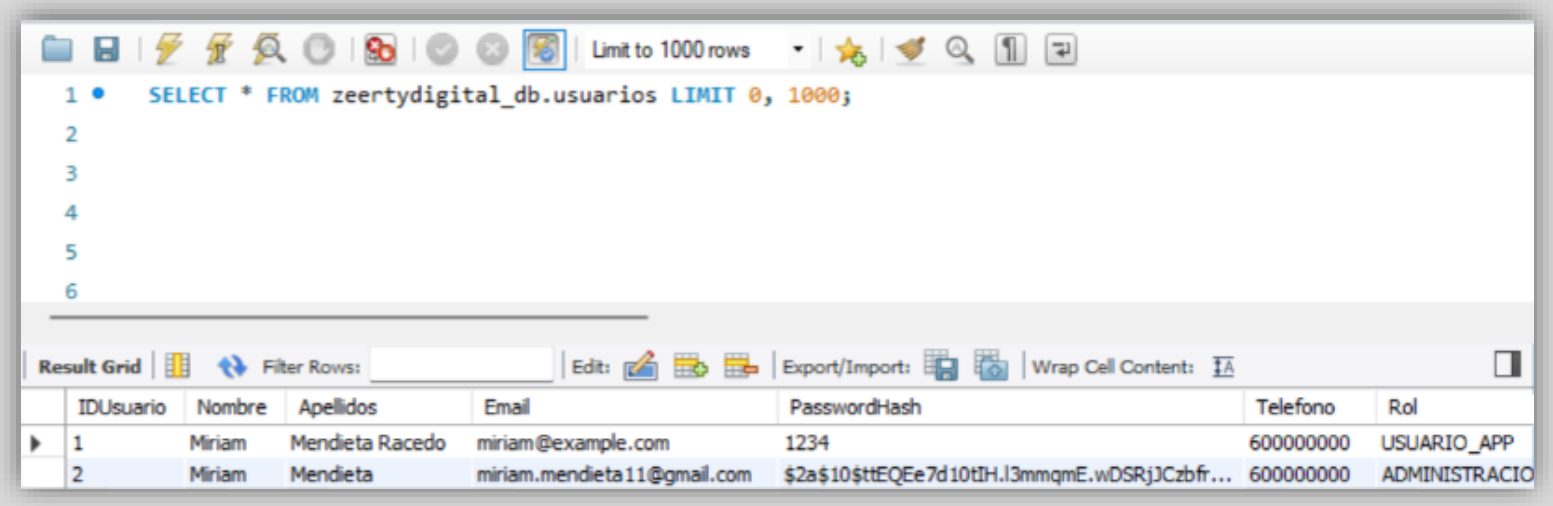
1. Levantar Spring Boot
2. Abrir el formulario de login
3. Introducir email y contraseña
4. Verificar que redirige a home.html





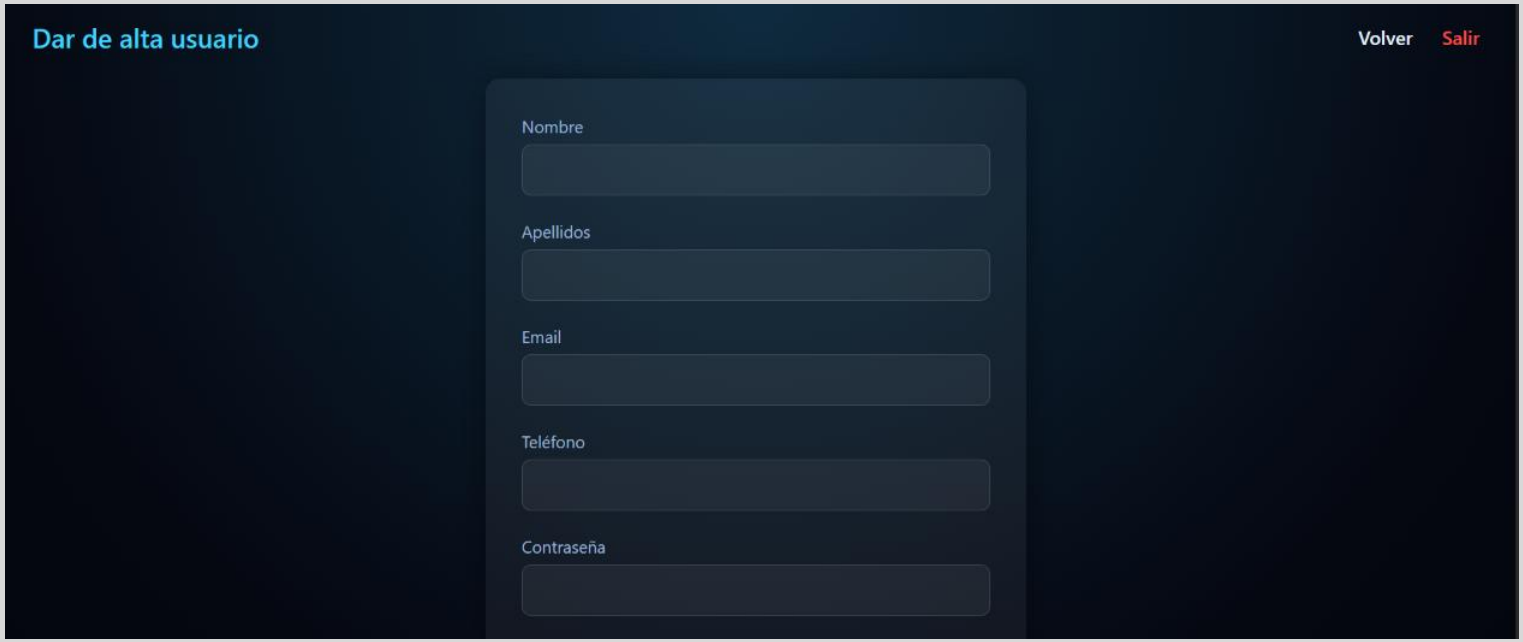
PROYECTO INTERMODULAR 2

```
2026-10-11T16:36:06.218+02:00  INFO 14616 --- [zeerty_digital_app] [  restartedMain] c.m.z.ZeertyDigitalAppApplication
     : Started ZeertyDigitalAppApplication in 8.816 seconds (process running for 9.724)
Hibernate: select u1_0.IDUsuario,u1_0.Apellidos,u1_0.Email,u1_0.Estado,u1_0.Nombre,u1_0.PasswordHash,u1_0.Rol,u1_0.Telefo
no from usuarios u1_0 where u1_0.Email=?
Hibernate: insert into usuarios (Apellidos,Email,Estado,Nombre,PasswordHash,Rol,Telefono) values (?,?,?,?,?,?,?)
Administrador creado correctamente
```



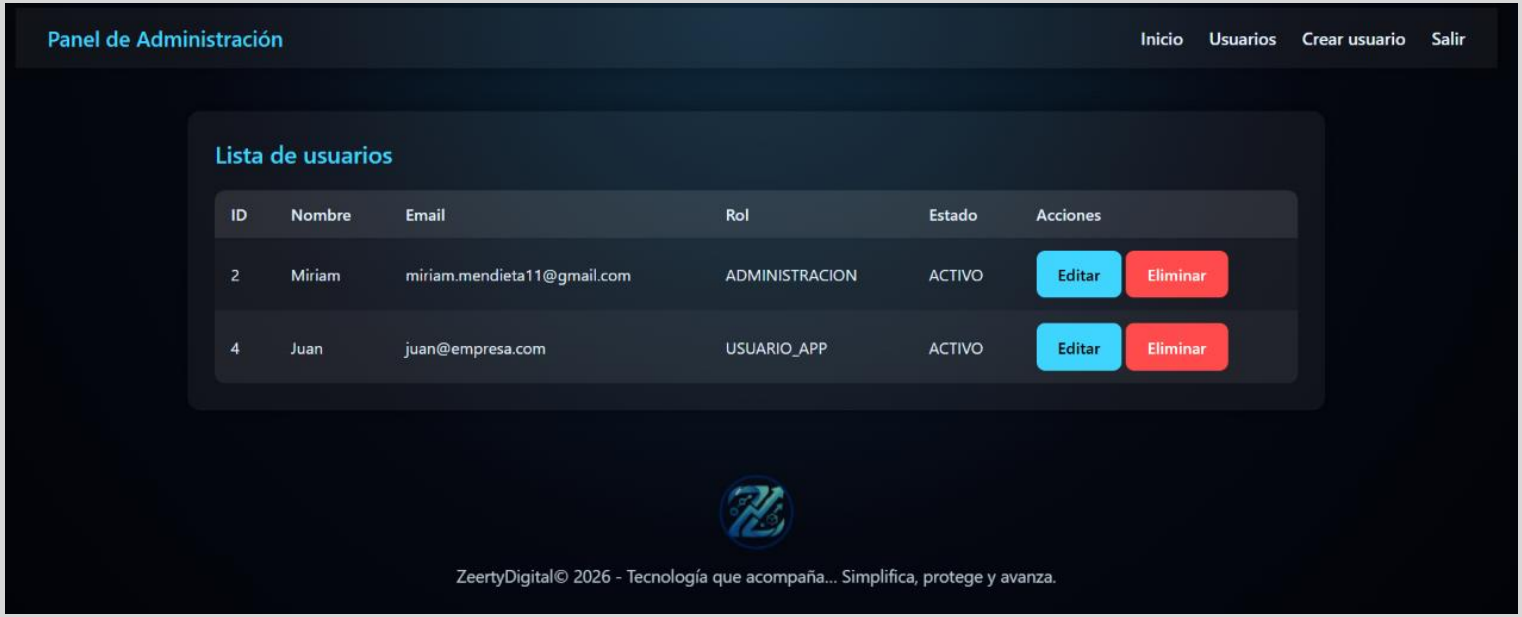
IDUsuario	Nombre	Apellidos	Email	PasswordHash	Telefono	Rol
1	Miriam	Mendieta Racedo	miriam@example.com	1234	600000000	USUARIO_APP
2	Miriam	Mendieta	miriam.mendieta11@gmail.com	$2a$10$ttEQEe7d10tIH.l3mmqmE.wDSRjJCzbfr...	600000000	ADMINISTRACIO


Panel de Administración
Bienvenid@, Miriam
Inicio
Usuarios
Crear usuario
Gestión de usuarios
Ver lista de usuarios
Dar de alta usuario
Ir a Home
Salir
ZeertyDigital© 2026 - Tecnología que acompaña... Simplifica, protege y avanza.



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

IDUsuario	Nombre	Apellidos	Email	PasswordHash	Telefono	Rol	Estado
2	Miriam	Mendieta	miriam.mendieta11@gmail.com	$2a$10$ttEQEe7d10tIH.l3mmqmE.wDSRjJCzbfr...	600000000	ADMINISTRACION	ACTIVO
4	Juan	Doe	juan@empresa.com	$2a$10$nf0by9OHCl9QS1yjlmjN3.7P5EYBeG5q...	123456789	USUARIO_APP	ACTIVO



ANEXOS

- METODOLOGÍA MoSCoW
- GLOSARIO
- APRENDIZAJE DE SPRING BOOT

BIBLIOGRAFÍA

- Apuntes del Profesor.
- Taller de SCRUM-PI2S. Septiembre 2026. Fco. Javier Tárrega Andreu.
- https://www.itdo.com/blog/moscow-que-es-y-como-priorizar-en-el-desarrollo-de-tu-aplicacion/
- https://www.youtube.com/watch?v=QMkRme6bEgE
- https://www.youtube.com/watch?v=lwNaN1vOKs4
- https://bonzzay.com/blog/stack-tecnologico-que-es-como-funciona/
- Curso UML: https://www.youtube.com/playlist?list=PLU8oAlHdN5BmmxXT0C2HO0bLRHZFWKbhH
- Editor de diagramas: http://dia-installer.de/download/index.html
- https://www.linkedin.com/pulse/5-tips-para-gestionar-mejor-tus-documentos-digitales-ipmconsulting-5lhyc/
- https://miro.com/es/diagrama/que-es-diagrama-uml/
- https://favicon.io/
- https://developer.mozilla.org/es/docs/Web/CSS/Guides/Animations/Using
- https://animista.net/play/basic/rotate
- https://lukehaas.me/projects/css-loaders/
- https://www.youtube.com/watch?v=5keD_70oQSg
- https://www.arquitecturajava.com/spring-boot-que-es/
- https://www.youtube.com/watch?v=C_6DqxKbGM8&t=1s
- https://spring.io/guides/gs/spring-boot
- https://gss-mtig.github.io/proy2Q/casos_uso/